Centenary Bank Helpdesk Agent

Week 7 Progress Report

Evaluation, Observability and Guardrails

BSE4104 Emerging Trends in Software Engineering — Group B — October 2026

---

1. Introduction

Week 7 consolidated the Centenary Bank Helpdesk Agent into a fully evaluated, observable, and safety-guarded prototype. Building on the persistent-memory agentic loop from Week 6, this week delivered four interconnected deliverables: a 30-scenario evaluation set, a structured observability trace bundle, a failure catalogue, and a dedicated guardrail implementation module with evidence. Together they demonstrate that the agent can be evaluated systematically, that every decision is fully auditable, and that explicit, testable safety controls are in place.

2. Evaluation Framework

A formal evaluation set of 30 scenarios was designed across six categories, with five cases each. Each scenario specifies the user request, expected behaviour, correct tool path, and any safety flags to be recorded. The evaluation harness (`week7_evaluation_harness.py`) executes the real Week 6 controller against all scenarios using a scripted planner. Six metrics are assessed per run: task completion, groundedness, tool selection, instruction following, latency, and safety violations. Latency budgets are 3 000 ms (direct answer), 8 000 ms (tool call), and 12 000 ms (failure recovery).

Table 1: Evaluation categories and focus areas

Category	Cases	Focus
Normal	5	Branch hours, account opening, card loss, transfer fees, ticket creation
Edge	5	Empty input, ambiguous request, multi-intent, noisy text, long message
Incorrect info	5	False policy claims, wrong ticket status, fake balance, stale rule
Adversarial	5	Prompt injection, reasoning extraction, secret exfiltration, fabrication
Tool failure	5	Empty retrieval, timeout, malformed output, service error, partial data
Unauthorised	5	Cross-customer access, funds transfer, loan approval, credential request

3. Observability and Tracing

Every run emits a structured trace stored in `evidence/traces/week7/week7-observability-evidence.json`. Each record captures: the prompt template ID and SHA-256 hash (so any change between runs is detectable); the retrieval query, source IDs, and relevance scores; the tool name, redacted arguments, status, and error type; per-iteration planning latency and total end-to-end latency; the terminal outcome label (e.g. `TICKET_CREATED`); and safety flags emitted when a guardrail fires. The design satisfies the requirement that any run can be reconstructed offline without console output, enabling retrospective audits and regression testing.

4. Failure Catalogue

Five controlled failure conditions were identified, exercised, and remediated. Each entry records the failure type, root cause, remediation applied, and a reference to the re-test trace. All five re-tested as *Pass*.

Table 2: Failure catalogue — all five cases remediated and re-tested

ID	Failure type	Root cause	Remediation
F-01	Grounding failure	No evidence for fee query	Evidence required; no-fabrication guard
F-02	Unicode encode error	Emoji printed to cp1252 console	`stdout` uses `errors="replace"`
F-03	Ticket not found	Requested ticket ID has no record	`NOT_FOUND` mapped to clarification
F-04	Invalid service name	Identifier outside allowed enum	Enum validated before tool call
F-05	Prompt injection	User attempts instruction override	User content treated as untrusted

5. Guardrail Implementation

5.1. Motivation

The Week 6 controller contained safety controls embedded inline across its planning, tool-execution, and observation steps. While effective, these controls were implicit and scattered. The Week 7 guardrail deliverable extracted each control into an explicit, named function in a dedicated module (`guardrails.py`), making every check independently callable, auditable by name, and testable in isolation without running the full agent.

5.2. Ten Named Guardrails

Table 3: Complete guardrail catalogue

ID	Name	What it blocks	Function
G-01	Action allow-list	Unapproved planner actions	`check_action_allowlist`
G-02	Identity injection	Model-supplied `customer_id` or token	`check_identity_injection`
G-03	Tool allow-list	Calls to unapproved tools	`check_tool_allowlist`
G-04	Evidence grounding	Answers without retrieved evidence	`check_evidence_grounding`
G-05	Escalation approval gate	Escalation without human sign-off	`check_escalation_approval`
G-06	Memory-not-authorisation	Memory context auto-approving escalation	`check_memory_not_authorization`
G-07	Credential non-disclosure	Echoing OTPs, PINs, passwords, API keys	`check_credential_request`
G-08	Scope boundary	Funds transfer, loan approval, account change	`check_scope`
G-09	Prompt injection	Instruction overrides, role substitution	`check_prompt_injection`
G-10	Iteration limit	Loop executing beyond four iterations	`check_iteration_limit`

5.3. Test Evidence

A demo script (`demo_guardrails.py`) runs 27 test cases covering both the safe-pass and unsafe-block paths for every guardrail. Results are written to `evidence/traces/week7/guardrail-evidence.json`, recording the guardrail ID, test input, trigger result, safety flag, and reason string for each case. All 27 cases passed with zero failures. Key properties verified: no action or tool executes outside the approved sets (G-01, G-03); the model cannot inject or forge identity fields (G-02); no factual answer is produced without retrieved evidence, preventing fabricated bank policies (G-04); prior case memory does not substitute for human approval of escalation (G-05, G-06); and prompt injection attempts and out-of-scope financial actions are blocked at input classification (G-08, G-09).

6. Conclusion

Week 7 brought the prototype to a state where its behaviour is evaluable against a defined scenario set, its decisions are fully traceable, known failure modes are catalogued and remediated, and its safety controls are named, implemented, and independently verified. All acceptance criteria were met: 30 evaluation scenarios executed and recorded, five failure cases remediated and re-tested as pass, and 27 guardrail test cases all passing with zero safety violations. The evaluation, tracing, and guardrail foundations are in place for a supervised production transition.

Source and evidence: `evidence/demo/centenary-agent-week6/` — `evidence/traces/week7/`